

MACHINE LEARNING · BEGINNER MANUAL

처음부터 끝까지 머신러닝 실전 입문

CSV 데이터를 받은 순간부터 문제 정의, 전처리, 모델 학습·평가·개선, 저장과 새 데이터 예측까지 이어지는 작업 매뉴얼

수학 초보자를 위한 예시·코드 중심 가이드
회귀·분류 상세 교재 연계

이 매뉴얼로 할 수 있는 것

처음 보는 표 데이터를 받았을 때 무엇부터 확인해야 하는지 순서대로 판단할 수 있습니다.

완료 기준

CSV를 불러와 X와 y를 정하고, 학습·시험 데이터를 분리하고, 전처리와 모델을 학습한 뒤 올바른 지표로 평가하고 파일로 저장할 수 있습니다.

전체 목차

1부	환경과 프로젝트 준비
2부	문제 정의와 데이터 이해
3부	데이터 정리와 전처리
4부	학습·시험 분리와 기준 모델
5부	회귀·분류 모델 학습과 평가
6부	교차검증, 모델 개선과 오류 분석
7부	저장, 재사용, 재현성과 실전 운영
부록	완성 코드, 문제 해결표, 최종 체크리스트

연계 교재

알고리즘의 상세 원리는 `regression_study_guide.pdf`와 `classification_study_guide.pdf`에서 다룹니다. 이 문서는 전체 프로젝트를 실제로 완성하는 순서에 집중합니다.

개발환경 준비

프로젝트마다 독립된 가상환경을 사용하면 라이브러리 버전 충돌을 줄일 수 있습니다.

```
# PowerShell: 프로젝트 폴더로 이동
cd C:\work

# 가상환경 생성과 활성화
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 필수 라이브러리 설치
python -m pip install --upgrade pip
pip install numpy pandas scikit-learn matplotlib seaborn jupyter joblib
```

라이브러리	주요 역할
NumPy	숫자 배열, 난수, 수치 계산
Pandas	CSV와 표 데이터 처리
Scikit-learn	전처리, 회귀·분류, 평가, 교차검증
Matplotlib-Seaborn	분포, 예측, 혼동 행렬 시각화
Jupyter	셀 단위 실습 문서
joblib	학습 모델과 Pipeline 저장

중요
pip 대신 python -m pip를 사용하면 현재 활성화된 파이썬 환경에 설치하는지 더 명확하게 확인할 수 있습니다.

권장 프로젝트 구조

```
my-ml-project/
├ data/
│   ├── raw/          # 원본, 직접 수정하지 않음
│   └── processed/     # 정리된 데이터
├ notebooks/          # 탐색과 실험
├ src/                 # 재사용할 파이썬 코드
├ models/              # 저장된 모델
├ reports/             # 그래프와 보고서
├ requirements.txt     # 라이브러리 버전
└ README.md            # 실행 방법과 목표
```

파일 관리 원칙

- 원본 데이터는 보존하고 정리 결과를 별도 파일로 만듭니다.
- 절대경로를 코드 곳곳에 쓰기보다 프로젝트 기준 경로를 정합니다.
- 데이터 파일명, 타겟 열, 단위를 README에 기록합니다.
- 비밀번호·개인정보·API 키는 노트북이나 저장소에 넣지 않습니다.

현재 C:Wwork 프로젝트

dataset에 CSV, supervisedLearning에 노트북과 교재가 있습니다. 현재 구조를 유지하되 노트북의 ./dataset와 ../dataset 경로는 실행 위치에 맞춰 통일해야 합니다.

모델보다 먼저 문제를 정의한다

좋은 모델보다 올바른 질문이 먼저입니다.

질문	결정할 내용	예시
무엇을 예측하는가?	타겟 y	내일 매출
누구의 어떤 시점을 예측하는가?	데이터 한 행의 의미	매장별 하루
언제 예측하는가?	예측 시점에 사용 가능한 x	전날 밤
어떤 실수가 위험한가?	평가 지표	매출을 크게 낮게 예측
결과를 어떻게 사용할가?	의사결정	재고와 근무 인원 결정

미래 정보 누수

내일 매출을 예측하면서 내일 실제 방문객 수처럼 예측 시점에 알 수 없는 정보를 x 로 사용하면 현실에서는 사용할 수 없는 모델이 됩니다.

문제 정의 문장

“전날까지 알 수 있는 기온·예보·요일·과거 매출을 사용해 매장별 다음 날 매출을 예측하고, RMSE와 MAE로 평가한다.”처럼 작성합니다.

회귀인가, 분류인가?

타겟 y	문제	대표 지표
가격, 점수, 시간, 수량	회귀	MAE, RMSE, R ²
합격/불합격, 스팸/정상	이진 분류	정확도, 정밀도, 재현율, F1
붓꽃 3종, 상품 카테고리	다중 분류	혼동 행렬, macro F1

같은 학생 데이터
"몇 점인가?"는 회귀, "합격인가?"는 분류입니다. x가 아니라 y의 의미로 문제 유형을 결정합니다.

성공 기준도 함께 정한다

- 집값 MAE가 현재 수작업 예측보다 작아야 함
- 암 선별 모델은 정확도보다 환자를 놓치지 않는 재현율을 우선
- 스팸 필터는 정상 메일을 보호하기 위해 스팸 정밀도를 우선

CSV를 불러오면 가장 먼저 볼 것

```
from pathlib import Path
import pandas as pd

path = Path("data/raw/sales.csv")
df = pd.read_csv(path)

print(df.shape)      # 행과 열 개수
print(df.head())     # 처음 5행
print(df.info())     # 열 이름, 타입, 결측치
print(df.describe()) # 숫자 열의 범위와 분포
print(df.nunique())  # 열별 고유값 개수
```

확인	질문
행	한 행은 고객인가, 거래인가, 하루인가?
열	예측 시점에 실제로 사용할 수 있는 정보인가?
타입	숫자가 문자로 읽히지 않았는가?
범위	나이가 -3, 점수가 900처럼 불가능한 값이 있는가?
고유값	ID처럼 거의 모든 행이 다른 열인가?

원칙

모델을 실행하기 전에 데이터 한 행과 각 열의 의미를 말로 설명할 수 있어야 합니다.

X와 y 분리, 사용하면 안 되는 열

```
target = "sales"
X = df.drop(columns=[target])
y = df[target]

print(X.shape)
print(y.shape)
```

제외를 검토할 열

- **단순 ID:** 고객번호·거래번호는 관계가 없는데도 모델이 외울 수 있습니다.
- **타겟의 다른 표현:** 총매출을 예측하면서 세금 포함 총액을 X로 쓰면 답을 미리 제공하는 셈입니다.
- **예측 이후 생성되는 정보:** 연체 여부를 예측하면서 연체 후 추심 결과를 사용하면 안 됩니다.
- **개인정보:** 목적에 꼭 필요하고 적법한지 검토합니다.

상관관계가 높다고 무조건 사용하지 않기

타겟과 매우 높은 관계가 보이면 좋은 특성일 수도 있지만, 타겟에서 파생되었거나 미래 정보가 섞인 누수일 수도 있습니다.

결측치, 중복, 이상치

```
print(df.isna().sum())  
print("duplicates:", df.duplicated().sum())
```

```
# 예: 가능한 범위 확인  
print(df["age"].min(), df["age"].max())
```

문제	가능한 처리	주의
결측치	행 제거, 중앙값·최빈값 대체	결측 자체가 중요한 신호일 수 있음
중복	진짜 중복이면 제거	같은 고객의 정상적인 반복 거래일 수 있음
이상치	오류 수정, 제외, 별도 분석	실제 희귀 사례를 무조건 삭제하지 않음

학습 데이터에서만 기준 계산

중앙값으로 결측치를 채울 때도 학습 데이터의 중앙값을 구하고 시험 데이터에는 같은 값을 적용해야 합니다.

문자 범주를 숫자로 바꾸기

지역·요일·상품 종류는 단순히 1,2,3으로 바꾸면 잘못된 크기 관계가 생길 수 있습니다.

지역	강남	마포	송파
강남	1	0	0
마포	0	1	0
송파	0	0	1

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
```

`handle_unknown="ignore"`는 학습 때 없던 지역이 새 데이터에 등장해도 즉시 오류가 나지 않도록 합니다. 다만 새로운 범주가 자주 생긴다면 데이터 정책을 점검해야 합니다.

순서가 있는 범주

낮음·중간·높음처럼 실제 순서가 있는 값은 순서 정보를 반영할 수 있습니다. 지역처럼 순서가 없는 범주와 구분합니다.

전처리보다 먼저 데이터 분리

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

학습 데이터

결측치 기준, 표준화 기준, 어휘, 모델 계수를 학습합니다.

시험 데이터

모든 선택이 끝난 뒤 처음 보는 데이터처럼 최종 평가합니다.

분류에서는 클래스 비율 유지

```
train_test_split(
    X, y, test_size=0.2,
    random_state=42,
    stratify=y
)
```

시간 데이터

미래를 예측하는 데이터는 무작위로 섞지 않고 과거로 학습하고 이후 시점으로 시험해야 합니다.

Pipeline으로 누수 없는 전처리

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LinearRegression

numeric = ["temperature", "visitors", "ad_cost"]
categorical = ["is_weekend", "region"]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

category_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric),
    ("cat", category_pipe, categorical)
])

model = Pipeline([
    ("preprocess", preprocess),
    ("model", LinearRegression())
])
```

Pipeline에 원본 DataFrame을 전달하면 학습 시 전처리 기준과 모델을 함께 학습하고, 예측 시 같은 처리를 자동 적용합니다.

복잡한 모델보다 기준 모델 먼저

새 모델이 실제로 유용한지 비교할 최소 기준이 필요합니다.

문제	간단한 기준
회귀	모든 사례에 학습 데이터 평균 또는 중앙값 예측
분류	가장 많은 클래스만 예측

```
from sklearn.dummy import DummyRegressor, DummyClassifier
```

```
baseline = DummyRegressor(strategy="mean")  
# baseline = DummyClassifier(strategy="most_frequent")  
baseline.fit(X_train, y_train)
```

판단

복잡한 모델이 기준 모델보다 좋아지지 않는다면 데이터, 특성, 문제 정의 또는 평가 방법부터 다시 확인합니다.

회귀 학습과 평가

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)
```

지표	읽는 방법
MAE	평균적으로 원래 단위에서 얼마나 빗나갔는가
RMSE	큰 실수를 더 강조한 오차 규모
R ²	평균만 예측하는 기준보다 흐름을 얼마나 잘 설명하는가

함께 확인

실제값-예측값 그래프와 잔차 그래프로 곡선, 이상치, 값이 커질수록 커지는 오차를 확인합니다.

분류 학습과 평가

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

지표	중심 질문
정확도	전체 중 몇 개를 맞혔는가?
정밀도	양성이라고 한 판단을 얼마나 믿을 수 있는가?
재현율	실제 양성을 얼마나 놓치지 않았는가?
F1	정밀도와 재현율이 함께 균형적인가?

불균형 데이터

다수 클래스만 맞혀도 정확도가 높을 수 있습니다. 혼동 행렬과 클래스별 지표, macro 평균을 함께 확인합니다.

모델 선택 빠른 안내

목적	우선 시도할 모델	이유
숫자 관계와 계수 해석	Linear/Ridge	단순하고 기준 모델로 좋음
표 데이터의 복잡한 관계	Random Forest	전처리가 비교적 단순하고 안정적
이진 가능성과 방향 해석	Logistic Regression	확률과 계수 제공
비슷한 이웃 기반	KNN	직관적, 표준화 필요
판단 경로 설명	Decision Tree	질문 구조 시각화
텍스트 기준 모델	TF-IDF + Naive Bayes/SVM	희소한 단어 특성에 적합

원칙

한 모델을 믿기보다 같은 분할과 평가 지표로 기준 모델과 2~3개 후보를 비교합니다.

교차검증으로 안정성 확인

한 번의 학습·시험 분할 결과는 우연히 좋거나 나쁠 수 있습니다. 학습 데이터를 여러 묶음으로 나누어 번갈아 검증합니다.

```
from sklearn.model_selection import cross_validate

scores = cross_validate(
    model, X_train, y_train,
    cv=5,
    scoring=["neg_mean_absolute_error", "r2"]
)
```

- 평균 점수: 일반적인 성능 수준
- 분할별 차이: 데이터에 따라 성능이 얼마나 흔들리는지
- 시험 데이터: 모델·설정 선택이 끝난 뒤 마지막 한 번 평가

전처리 포함

교차검증에는 전처리와 모델이 묶인 Pipeline 전체를 전달해야 각 검증 묶음의 정보가 학습에 새지 않습니다.

하이퍼파라미터 조정

```
from sklearn.model_selection import GridSearchCV

params = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [None, 5, 10]
}

search = GridSearchCV(
    model, params,
    cv=5,
    scoring="neg_mean_absolute_error"
)
search.fit(X_train, y_train)
best_model = search.best_estimator_
```

지표는 목적에 맞게

회귀에서 큰 실수가 위험하면 RMSE, 평균적인 금액 차이가 중요하면 MAE를 사용합니다. 분류에서 환자 놓침이 위험하면 재현율, 균형이 중요하면 F1을 선택합니다.

많이 시험할수록 좋은 것은 아님

근거 없이 거대한 조합을 탐색하면 시간만 늘고 검증 데이터에 과적합될 수 있습니다. 먼저 중요한 설정과 현실적인 범위를 정합니다.

점수 다음에는 틀린 사례를 본다

회귀

- 오차가 가장 큰 행을 정렬해 공통점을 찾습니다.
- 특정 지역·가격대·시기에서 계속 낮게 예측하는지 확인합니다.
- 잔차가 곡선이나 깔때기 모양인지 봅니다.

분류

- FP와 FN 사례를 따로 확인합니다.
- 소수 클래스가 주로 틀리는지 봅니다.
- 업무상 더 위험한 실수를 줄이도록 임계값과 지표를 조정합니다.

```
# 회귀 오차가 큰 사례
result = X_test.copy()
result["actual"] = y_test
result["predicted"] = y_pred
result["abs_error"] = (result["actual"] - result["predicted"]).abs()
print(result.sort_values("abs_error", ascending=False).head(10))
```

개선의 출발점

평균 점수를 조금 높이는 것보다 실제로 어떤 데이터에서 왜 실패하는지 이해하는 것이 더 유용할 수 있습니다.

전처리와 모델을 함께 저장

```
import joblib

# Pipeline 전체 저장
joblib.dump(best_model, "models/sales_model.joblib")

# 다시 불러오기
loaded_model = joblib.load("models/sales_model.joblib")
prediction = loaded_model.predict(new_data)
```

스케일러나 OneHotEncoder를 따로 저장하지 않고 Pipeline 전체를 저장하면 새 데이터에도 학습 때와 같은 순서가 자동 적용됩니다.

함께 기록할 것

- 모델 파일 생성 날짜와 버전
- 사용한 데이터 기간과 타겟 정의
- 입력 열 이름, 순서, 타입, 단위
- 라이브러리 버전
- 시험 데이터 평가 결과
- 알려진 한계와 사용하면 안 되는 상황

보안

신뢰할 수 없는 출처의 joblib-pickle 모델 파일은 불러오지 않습니다. 로딩 과정에서 임의 코드가 실행될 수 있습니다.

새 데이터 예측 전 검증

```
required = ["temperature", "visitors", "ad_cost", "is_weekend", "region"]

missing = set(required) - set(new_data.columns)
if missing:
    raise ValueError(f"Missing columns: {sorted(missing)}")

prediction = loaded_model.predict(new_data[required])
```

확인사항

- 학습 때와 같은 열 이름과 의미인가?
- 원과 만원, m와 km처럼 단위가 바뀌지 않았는가?
- 숫자 열이 문자열로 들어오지 않았는가?
- 학습 범위를 크게 벗어난 값이 아닌가?
- 새로운 범주와 결측치를 Pipeline이 처리할 수 있는가?

예측값은 사실이 아님

모델 출력은 과거 데이터에서 학습한 추정치입니다. 중요한 의사결정에서는 입력 범위, 불확실성, 사람의 검토 절차를 함께 둡니다.

같은 결과를 다시 만드는 기록

```
# 주요 무작위 설정 고정
```

```
random_state = 42
```

```
# 설치 버전 기록
```

```
pip freeze > requirements.txt
```

기록 항목	예시
실험 이름	sales_rf_v3
데이터	2025-01~2025-12, 18,420행
분할	시간 기준, 마지막 2개월 시험
특성	기온, 요일, 과거 7일 매출
모델·설정	RandomForest, depth=10
결과	MAE 82만원, RMSE 121만원

Seed의 한계

Random Seed는 같은 무작위 순서를 재현하도록 돕지만 데이터·코드·라이브러리 버전이 달라지면 결과도 달라질 수 있습니다.

회귀 프로젝트 완성 템플릿

```
import joblib
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("data/raw/sales.csv")
X = df.drop(columns=["sales"])
y = df["sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

num = ["temperature", "visitors", "ad_cost"]
cat = ["is_weekend", "region"]

preprocess = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), num),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), cat)
])

model = Pipeline([
    ("preprocess", preprocess),
    ("model", Ridge(alpha=1.0))
])

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("MAE", mean_absolute_error(y_test, y_pred))
print("RMSE", mean_squared_error(y_test, y_pred) ** 0.5)
print("R2", r2_score(y_test, y_pred))

joblib.dump(model, "models/sales_model.joblib")
```

자주 발생하는 문제

현상	주요 원인	확인
FileNotFoundError	작업 폴더와 상대경로 불일치	os.getcwd(), 파일 존재
could not convert string to float	문자 범주 미처리	OneHotEncoder 사용
Input contains NaN	결측치 남음	SimpleImputer 사용
열 개수·이름 오류	학습·예측 입력 구조 다름	Pipeline과 required 열 확인
학습은 좋은데 시험은 나쁨	과적합, 누수, 분포 차이	교차검증, 복잡도 축소, 오류 분석
정확도만 매우 높음	클래스 불균형	혼동 행렬, recall, F1
실행마다 결과가 다름	Seed·분할·버전 차이	random_state와 버전 기록
SGD 수렴 경고	스케일, 학습률, 반복 횟수	표준화와 max_iter 점검

디버깅 순서

오류 메시지 마지막 줄 → 현재 작업 폴더 → 데이터 shape·열 이름·dtype → 결측치 → 학습/예측 전처리
일치 순으로 확인합니다.

프로젝트 완료 체크리스트

- ✓ 예측 시점, 타겟 y , 데이터 한 행의 의미를 문장으로 정의했다.
- ✓ 회귀·분류를 y 의 성격으로 선택했다.
- ✓ 예측 시점에 알 수 없는 열과 ID·타겟 누수 열을 제외했다.
- ✓ 결측치, 중복, 이상치, 타입과 단위를 확인했다.
- ✓ 전처리 전에 학습·시험 데이터를 분리했다.
- ✓ 전처리와 모델을 Pipeline으로 묶었다.
- ✓ 간단한 기준 모델보다 나은지 확인했다.
- ✓ 목적에 맞는 평가 지표를 선택했다.
- ✓ 교차검증으로 결과의 흔들림을 확인했다.
- ✓ 가장 크게 틀린 실제 사례를 분석했다.
- ✓ 시험 데이터는 모든 선택 후 마지막 평가에 사용했다.
- ✓ Pipeline 전체와 입력 스키마를 함께 저장했다.
- ✓ 데이터 기간, 코드, 라이브러리 버전, Seed와 결과를 기록했다.
- ✓ 모델의 한계와 사용하면 안 되는 상황을 문서화했다.

최종 원칙

좋은 머신러닝 프로젝트는 복잡한 모델보다 **명확한 문제, 누수 없는 데이터 처리, 공정한 평가, 재현 가능한 기록**에서 시작합니다.